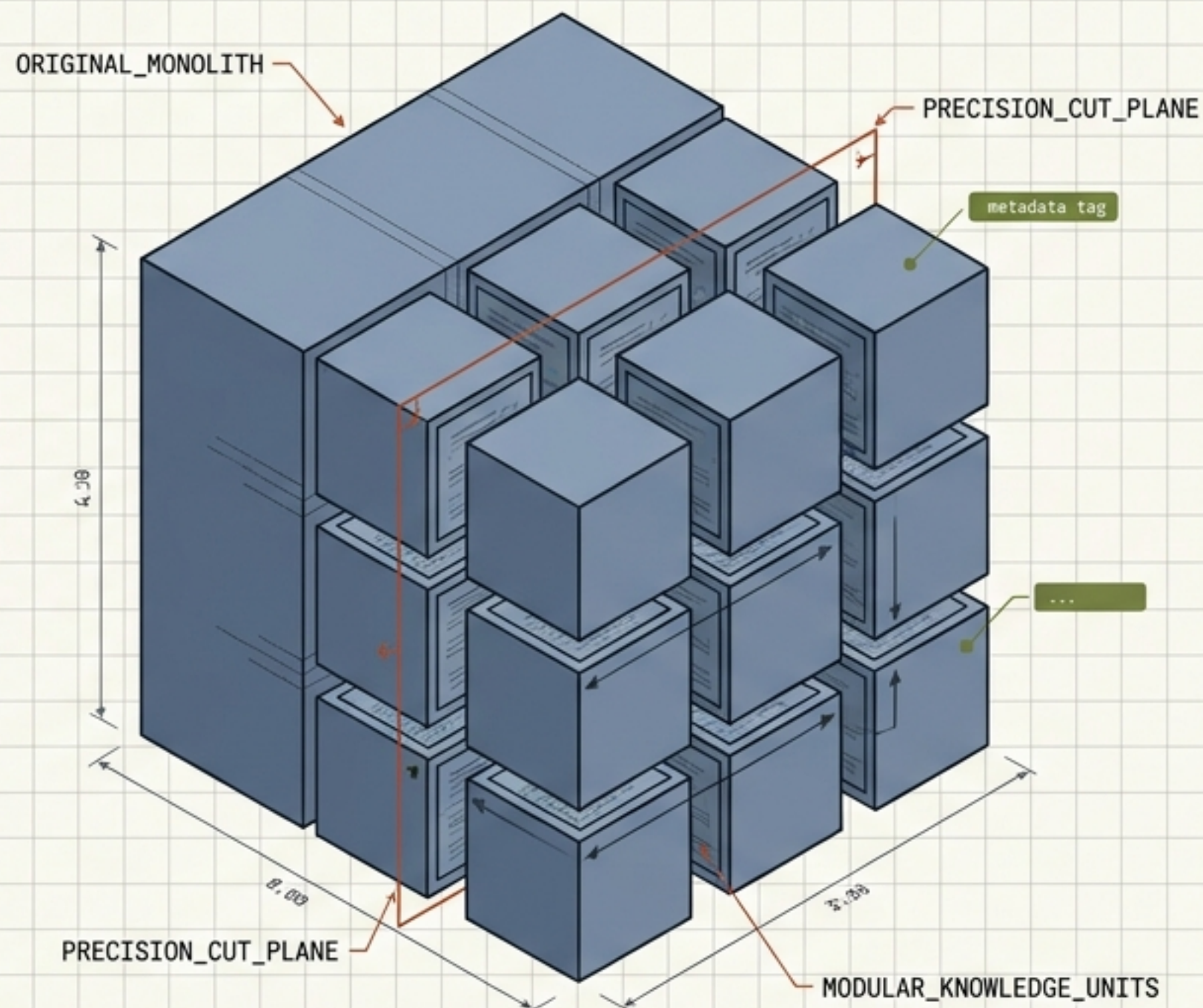


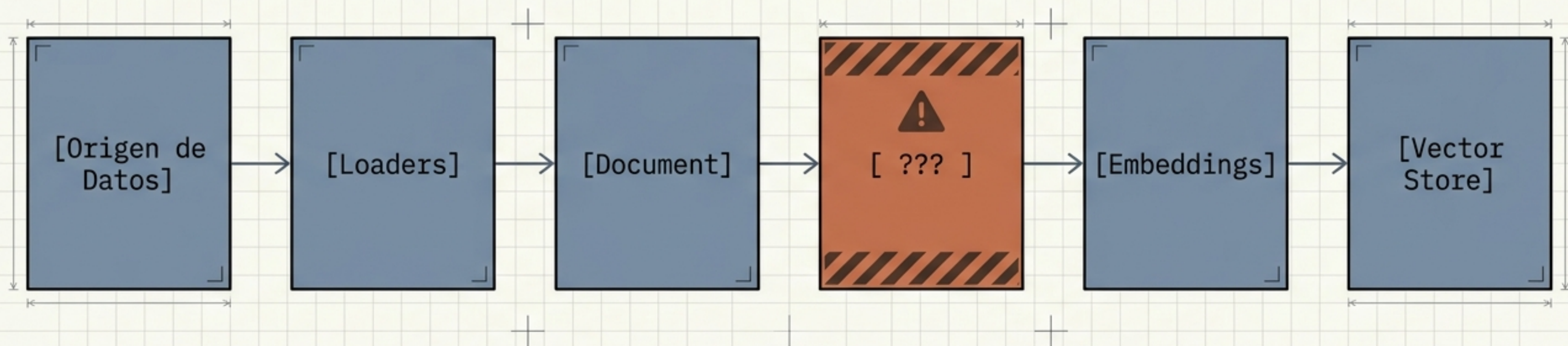
Ingeniería de RAG: El Arte y Ciencia del Chunking

Diseñando unidades de
conocimiento con
LangChain

MÓDULO DE ARQUITECTURA DE DATOS Y BASES VECTORIALES



1. Dónde Estamos: El cuello de botella del pipeline



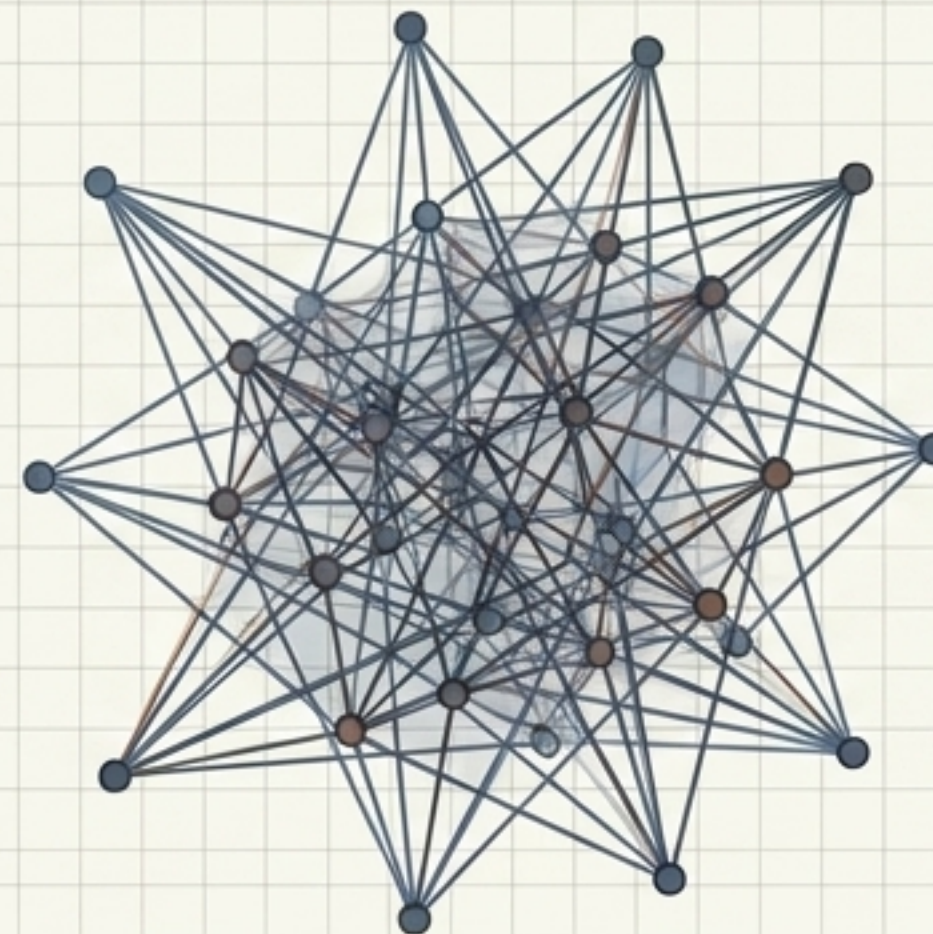
Hasta este momento, hemos extraído el texto en objetos Document. Pero un LLM no procesa 100 páginas de golpe, y los modelos de embedding tienen límites estrictos. Necesitamos transformar documentos monolíticos en fragmentos digeribles.

2. El Problema: La paradoja de la compresión conceptual

El Documento Gigante



El Espacio Vectorial



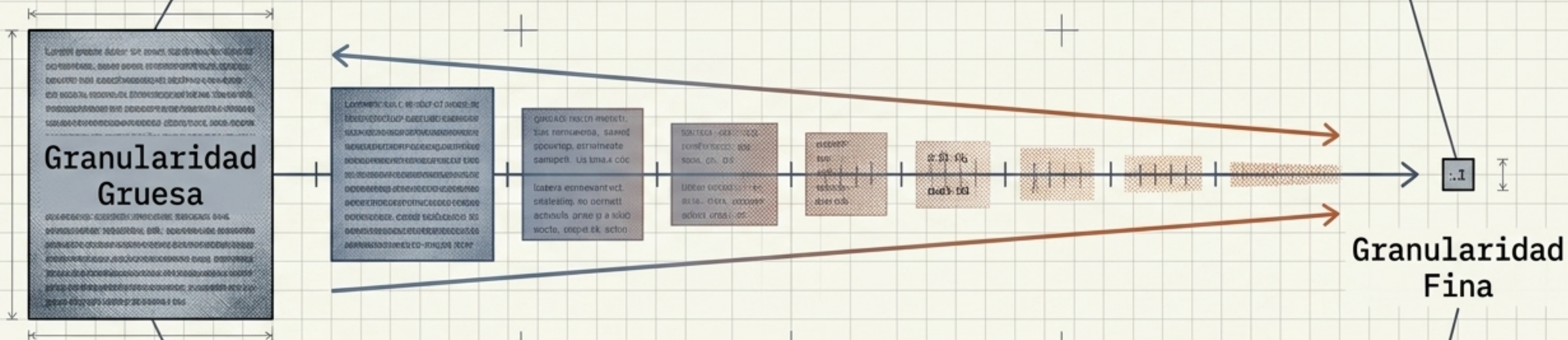
Si generamos un único embedding para todo un manual de 100 páginas, ese vector debe representar simultáneamente demasiados conceptos dispares. La búsqueda perderá especificidad y recuperará ruido.

¿Cómo construimos unidades de conocimiento más precisas?

3. Teoría: El *trade-off* de la Granularidad

Ventaja: Se conserva mucho contexto (ideas completas).

Ventaja: Concepto muy específico; el vector es preciso.

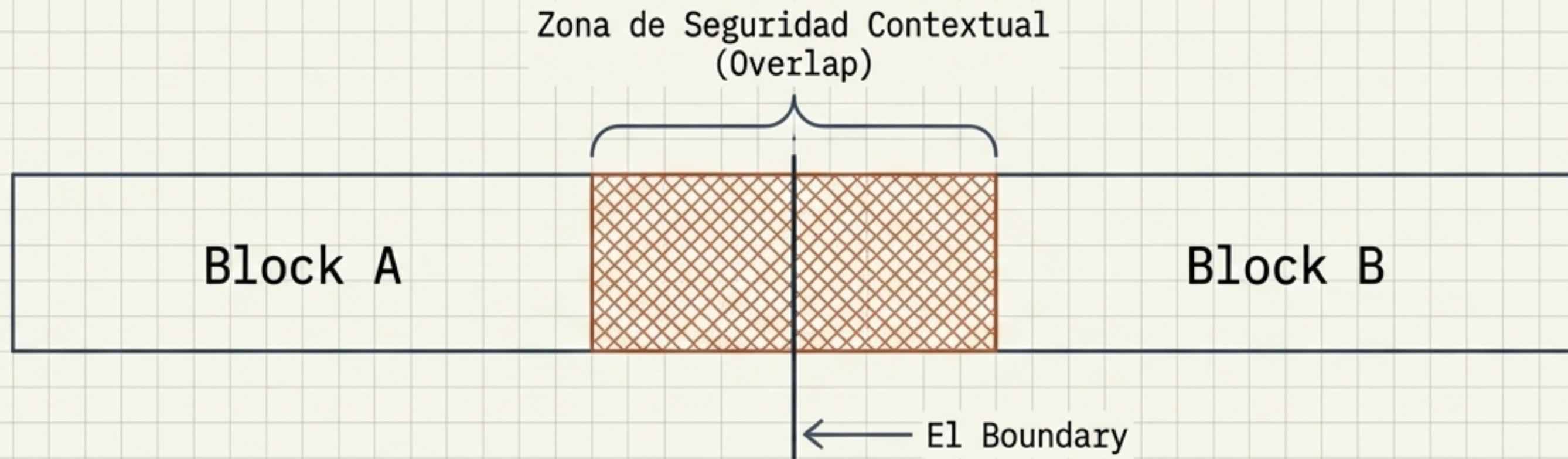


Desventaja: Contiene múltiples temas; el vector resultante es ambiguo y arrastra ruido al LLM.

El objetivo del chunking no es cortar caracteres, sino definir la unidad mínima recuperable del sistema.

Desventaja: Se pierde el contexto; el fragmento por sí solo (ej. "Él lo aprobó") es inútil.

El problema de los Boundaries y el Overlap



El Boundary

- El límite exacto donde termina un fragmento y empieza el otro. Si cortamos en medio de una oración, rompemos la unidad lógica.

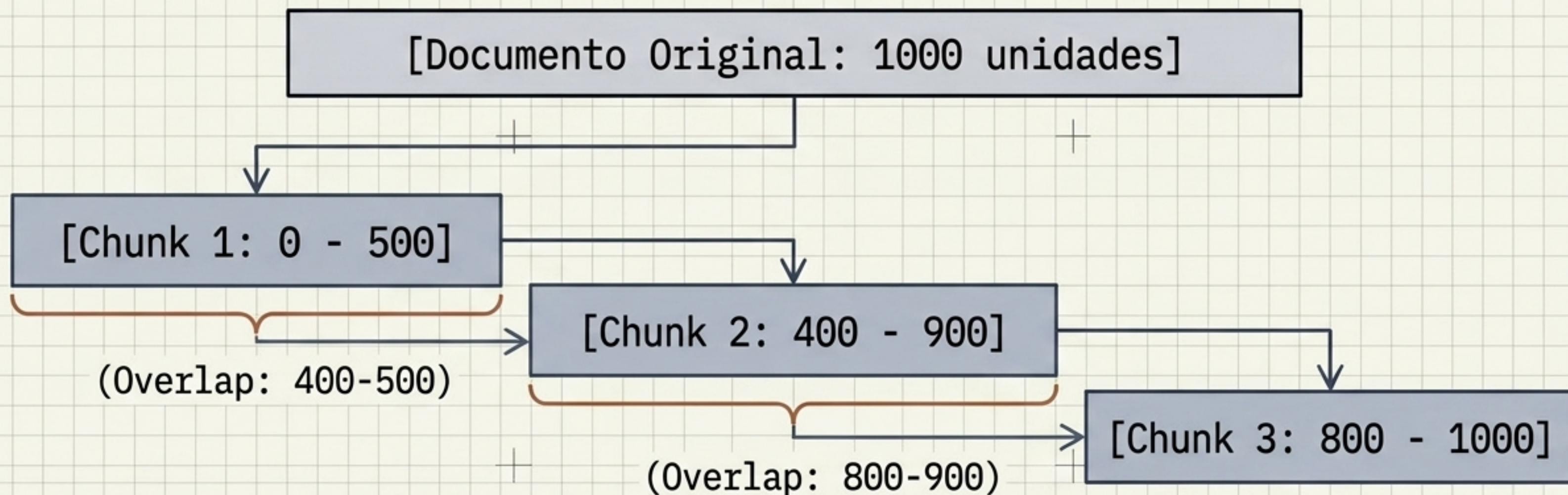
chunk_overlap

Permite que las ideas de las zonas fronterizas aparezcan en ambos fragmentos, evitando que una relación semántica se pierda al dividir el documento.

Advertencia

Más overlap = mayor redundancia, más chunks, más llamadas al embedding y mayor latencia.

4. Mecánica de partición con ventanas deslizantes



Cada nuevo chunk no avanza chunk_size completo,
sino $\text{chunk_size} - \text{chunk_overlap}$.

5. Alternativas de Chunking

Estrategia	Señal	Preserva Estructura	Uso Típico
Fixed-size	Longitud	No	Demos, baselines
Recursive	Jerarquía	Parcial	Texto general, RAG robusto
Structure/Markdown	Headers	Sí	Documentación técnica
Code-aware	Sintaxis (AST)	Sí	Code RAG
Semantic	Embeddings	Semántica	Casos muy complejos, alto costo

Para este proyecto utilizaremos una estrategia híbrida:
Estructural (Markdown) + Recursivo (Caracteres)
para preservar jerarquía y controlar el tamaño.

6. El estándar de la industria: RecursiveCharacterTextSplitter

Clase: `langchain_text_splitters.RecursiveCharacterTextSplitter`

Input: Objetos Document o string

Output: Lista de Documents

Jerarquía de separación

Párrafos (`\n\n`)

↓ Si es muy grande

Líneas (`\n`)

↓ Si es muy grande

Palabras ()

↓ Si es muy grande

Caracteres ()

Intenta dividir por párrafos.
Si el fragmento sigue siendo muy grande (recursividad), pasa al siguiente separador.

7. Código Mínimo: Instanciación

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Instanciamos el splitter base
recursive_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=75,
    length_function=len
)
```

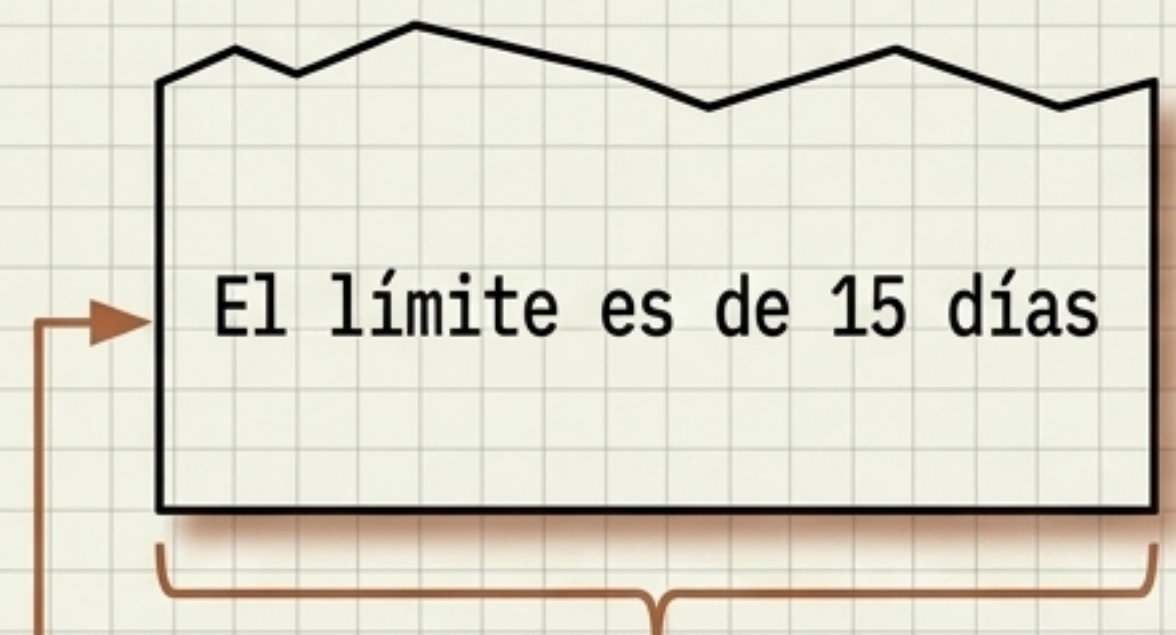
← No es absoluto. Es un objetivo que respeta límites lógicos.

← Zona de seguridad contextual.

← Aquí usamos len (caracteres), pero podría usarse tiktoken (tokens).

Evolución Estratégica: Contexto Estructural

El Problema del Splitter Puro Sin Estructura

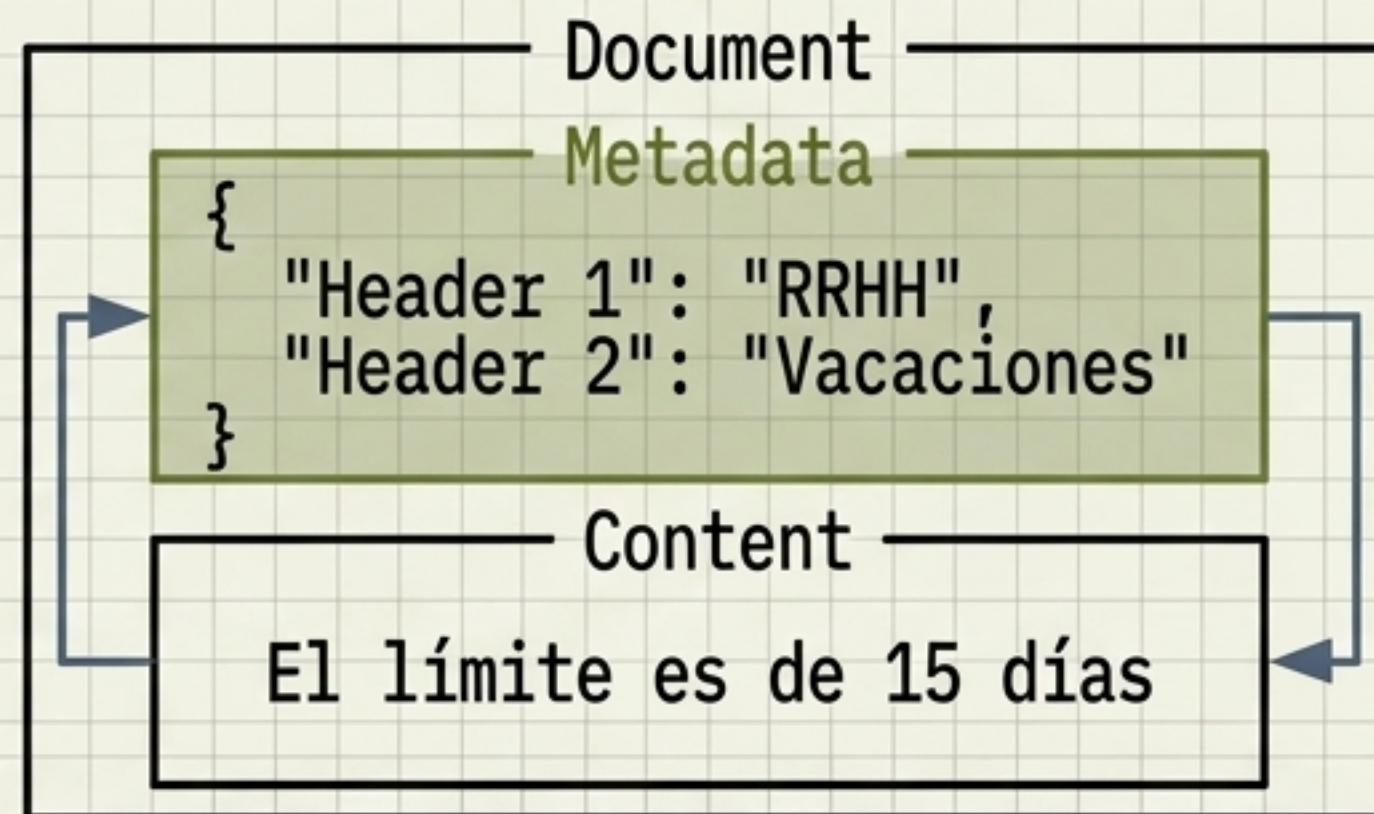


El límite es de 15 días

The diagram shows a simple rectangular box with a wavy top edge. A brown arrow points from the left towards the box. Below the box, a brown bracket spans its width.

(¿De qué? ¿Vacaciones? ¿Licencias?)

Structure-Aware Chunking Preservando Jerarquía



Ignorar los headers humanos es perder una señal semántica invaluable.
MarkdownHeaderTextSplitter extrae esta estructura hacia los metadatos
antes de medir el tamaño.

8. Integración: El Pipeline Híbrido (Markdown + Recursivo)

Stage 1: Estructura

```
from langchain_text_splitters import MarkdownHeaderTextSplitter
headers_to_split_on = [("#", "Header 1"), ("##", "Header 2")]
markdown_splitter = MarkdownHeaderTextSplitter(headers_to_split_on)

# 1. Preservamos jerarquía
md_docs = markdown_splitter.split_text(documento_completo)
```



Stage 2: Tamaño

```
# 2. Controlamos tamaño sobre los documentos ya estructurados
final_chunks = recursive_splitter.split_documents(md_docs)
```

Pasamos de texto plano (split_text) a objetos LangChain (split_documents) en un flujo de refinamiento continuo.

9. Inspección de Resultados: ¿Qué produce LangChain?

```
Document {  
  "page_content": "Todo empleado tiene  
derecho a 15 días hábiles...",  
  "metadata": {  
    "Header 1": "Políticas Corporativas",  
    "Header 2": "Vacaciones",  
    "source_doc_id": "manual_v3_2023",  
    "chunk_id": "c_004"  
  }  
}
```

`page_content`: Lo que enviaremos al modelo de Embedding.

`metadata`: Crucial para filtros posteriores (ej. buscar solo en '`Vacaciones`') y para trazabilidad (`chunk_id`).

Evaluación: El Chunking es un hiperparámetro medible



Nunca optimices tu chunking evaluando "a ojo" con una sola pregunta del LLM. Separa Retrieval de Generation.

Hit@K



¿El chunk que contiene la respuesta exacta apareció en el Top-K recuperado?

Recall



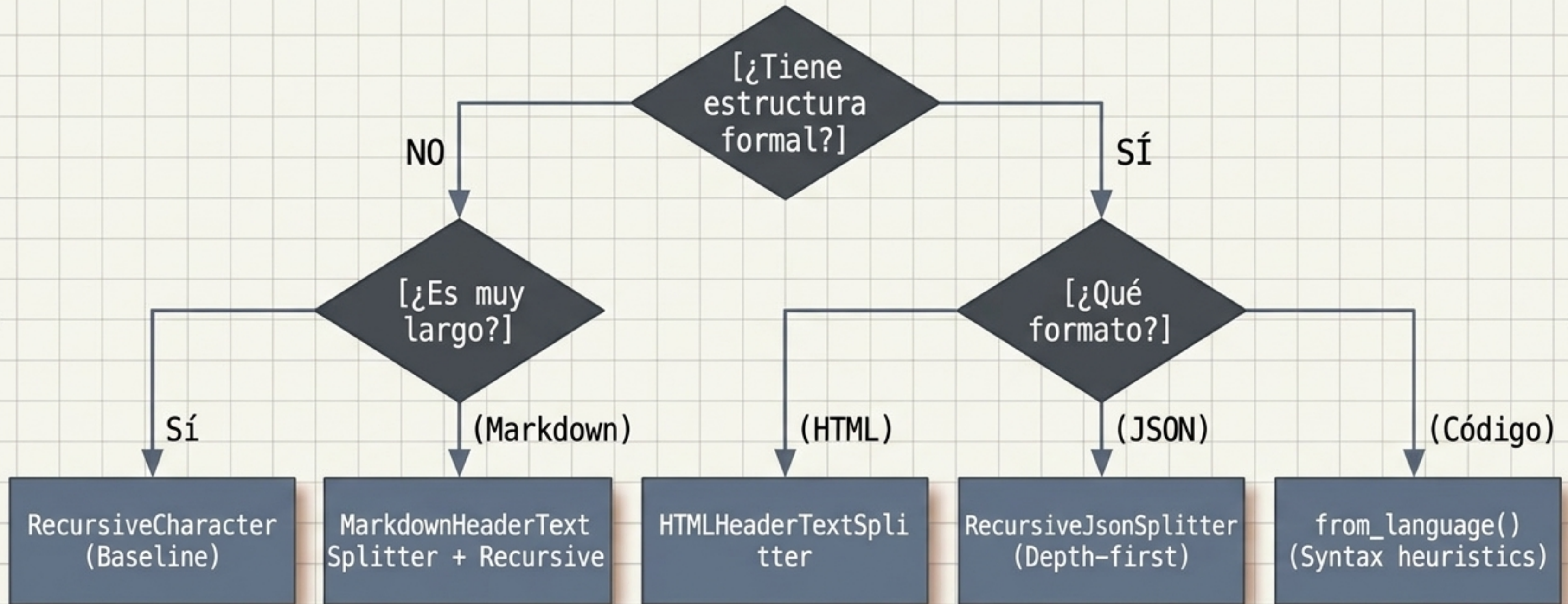
De toda la información relevante que debía recuperarse, ¿cuánta capturamos en los chunks?

Context Payload



¿Cuántos tokens basura/redundantes estamos enviando al LLM por un chunk_size excesivo?

Síntesis: El Árbol de Decisión del Chunking



Si los baselines fallan en medir cambios temáticos complejos, escalar a Semantic Chunking.

10. Pipeline Actualizado



“Chunking define qué puede recuperar el sistema. Retrieval define qué recupera. El LLM solamente puede razonar con aquello que finalmente recibe.”

Siguiente fase: Vectorización (Embeddings) y Búsqueda (K-NN, MMR).